

Lecture 1 • Mon 9/28/2026

CS160 Compilers — Fall 2026

The Big Picture.

Yu Feng

Laptops closed, please.

01

Yu Feng.

I build tools that read code more carefully than people do — and then prove what they found.

Research

Programming languages, formal methods and security: program synthesis, static analysis and verification — pointed at smart contracts and zero-knowledge proofs, where one missed bug costs real money.

Industry

Founder and technical leader behind several AI and blockchain startups now valued in the billions — taking the analyses from papers like the ones in this course and running them on code that ships.

yufeng@cs.ucsb.edu · office hours Mon 11–noon, HFH 2157 · TA: Hanzhi Liu, hanzhi@ucsb.edu

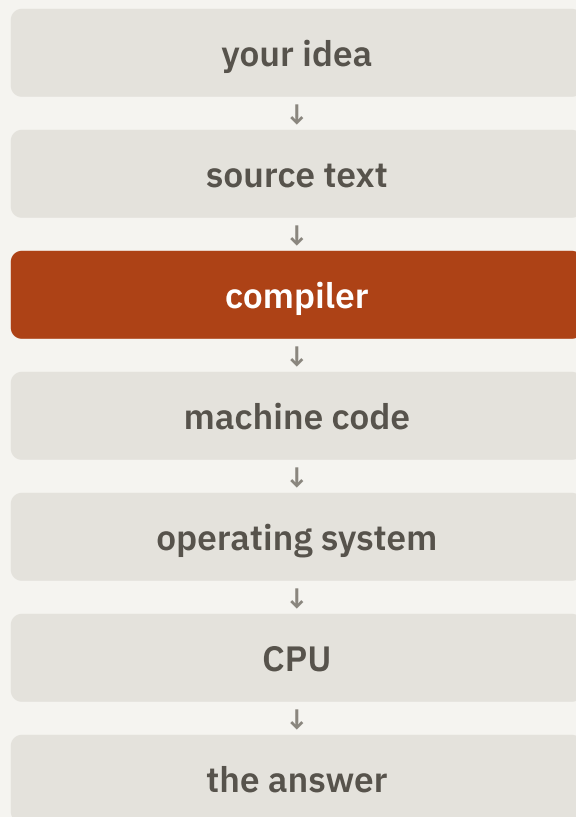
What happens between typing `print("Hello CS160!")` and seeing it on the screen?

Most programmers never look. This course is about looking.

What is a compiler?

**A program that translates a program
from one language into another.**

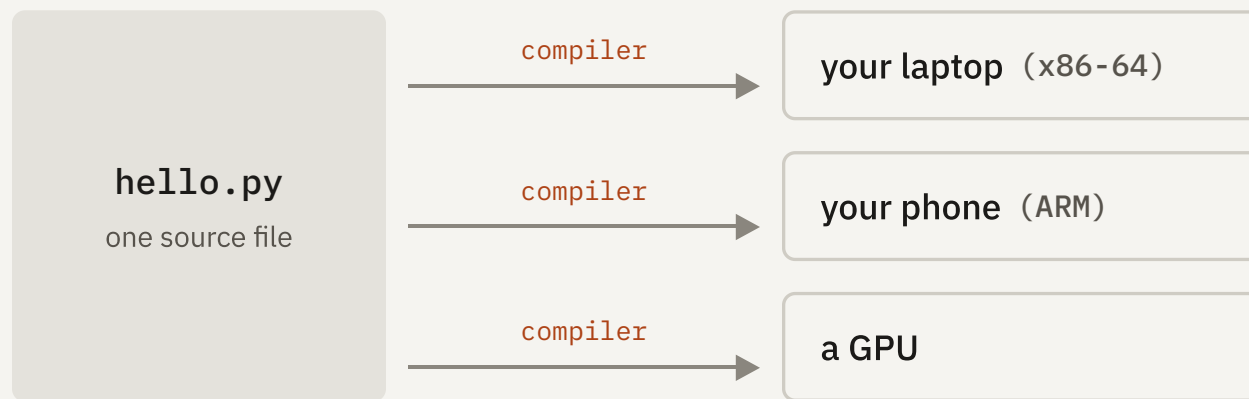
Source code in. Machine code out. Same behavior.



The life of a program.

Everything above the compiler is human. Everything below it is physics. The compiler is the translation between them.

Why you have never written machine code.



You write the program once, in a language made for people. The compiler writes it three more times, in languages made for chips.

Why compilers are hard — and beautiful.

They must be *correct on every program anyone will ever write* — not just the ones you tested.

They must be *fast*: a good optimizer finds a 100× speedup in code it has never seen.

And they must do both while never, ever changing the answer.

Almost every big idea in CS shows up inside one: automata, grammars, type theory, graph algorithms, fixpoints, the machine itself.

The same program, three ways.

One hundred million steps of a pseudorandom number generator — the same arithmetic `rand()` does — adding up the high bits as it goes.

```
repeat 100,000,000 times:
    x  = x * A + C      # A, C: the standard 64-bit LCG constants
    acc = acc + (x >> 33) # keep the high bits, sum them
print(acc)
```

	time	prints	
Python 3	19.06 s	107372821177053599	
C, clang -O0	0.31 s	107372821177053599	61× faster
C, clang -O2	0.14 s	107372821177053599	135× faster

Same algorithm. Same answer — look at the third column. Every difference is the compiler.

Would you have caught it?

```
// Prompt to an AI: "Write a C function that returns the average of an int array."  
int average(int *a, int n) {  
    int sum;  
    for (int i = 0; i <= n; i++)  
        sum += a[i];  
    return sum / n;  
}
```

Fifteen seconds.

Most of the code you ship will be written by a model.

It will be plausible.

It will sometimes be wrong.

Something has to check it and make it fast — and that something is a **compiler**.

Understanding the layer beneath generated code is how you stay in charge of it.

The password that was never erased.

```
int login(void) {  
    char password[64];  
    read_password(password);           // fill the buffer  
    int ok = check(password);  
    memset(password, 0, sizeof password); // wipe the secret before returning  
    return ok;  
}
```

The programmer was careful. The compiler was correct. The secret leaked anyway.

The compiler is the **trust boundary** between code and the machine.

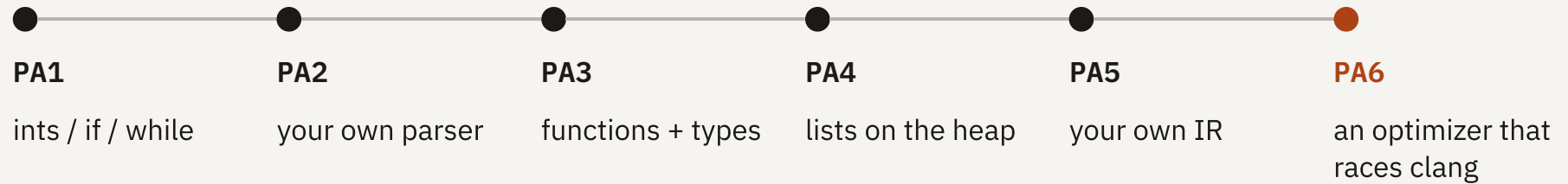
Above it, everything can be wrong. Below it, everything runs.

Understand it and you can follow any program from the idea to the CPU — and back.

That is what the next ten weeks are for.

One compiler. Ten weeks.

Python-like source in, native x86-64 out.



ChocoPy: the language you'll compile.

```
def is_even(n: int) -> bool:
    return n % 2 == 0

def count_evens(xs: [int]) -> int:
    count: int = 0
    i: int = 0
    while i < len(xs):
        if is_even(xs[i]):
            count = count + 1
        i = i + 1
    return count

print(count_evens([3, 8, 12, 5]))
```

A statically typed subset of Python — with a published language reference, which is your spec.

We start with a sliver of it and grow every week.

You already know how to read it.

How you're graded.

60%	Midterms — 3 × 20%, in class: 10/21, 11/9, 12/2
30%	Programming assignments — 6 × 5%
10%	Worksheets — one per lecture from Wednesday; 75% turned in = full credit
+2%	Extra credit — top-5 Slack participants

No final exam. Cheat sheet: one letter-size page, both sides.

Four rules.

- 1 Lectures are not recorded.
- 2 A worksheet every lecture (from Wednesday), turned in at the end.
- 3 No screens in lecture (DSP accommodations: see me this week).
- 4 Exams happen on the announced dates. No makeups. Plan travel now.

Use AI on the assignments. At your own risk.

You may use Copilot / ChatGPT / Claude on PAs.

If you do, submit your coding session with the PA — we grade it and give you feedback on how to use AI more effectively.

The exams are built from the PAs and are entirely analog; your work must be your own — **no shared solutions, no public repos, no previous quarters.**

Join tonight.



Slack · cs160-fall26

Install Python 3.12 and clang before Thursday's section — instructions pinned in Slack.

Lecture Mon/Wed

Section Thu 11:00–11:50am, BRDA 1640 · Hanzhi

Office hours Mon 11–noon, HFH 2157 · Yu

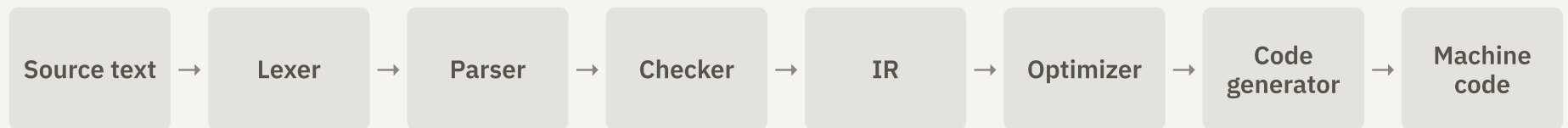
TA office hours Wed 11:00–11:50am, HFH 2152A · Hanzhi

Textbook Cooper & Torczon, *Engineering a Compiler*, 3e

Course website github.com/fredfeng/CS160

join.slack.com/t/cs160-fall26/shared_invite/zt-4akvdxrn-Stkjknk00q~7zuczYzt2nQ

The map.



Every compiler — clang, javac, V8, the one inside PyTorch — has this shape. This is the map we follow for ten weeks.

Front end: text becomes structure.



Lexer: characters → words (tokens).

Parser: words → a tree (structure).

Checker: a tree → a tree with types (meaning).

Weeks 3-6.

Middle end: structure becomes something the compiler can reason about.



IR: a small, simple language with none of the source's sugar.

Optimizer: rewrite the IR to run faster — without changing the answer.

Weeks 8–9. This is where slide 8's 135× came from.

Back end: down to the machine.



Code generator: IR → the instructions this CPU actually runs. Registers, calling conventions, memory layout.

Weeks 5 and 10.

One line, four forms.

```
x = 2 * (3 + 4)                                ← source
NAME(x) EQ NUM(2) STAR LPAREN NUM(3) PLUS NUM(4) RPAREN  ← tokens
Assign(x, Mul(2, Add(3, 4)))                    ← tree
%t1 = add i32 3, 4
%t2 = mul i32 2, %t1                            ← IR (you'll write this Wednesday)
```

Each form is the same program, one step closer to the machine.

Interpreter or compiler?

Interpreter

reads the program and ***does*** what it says.

program → answer

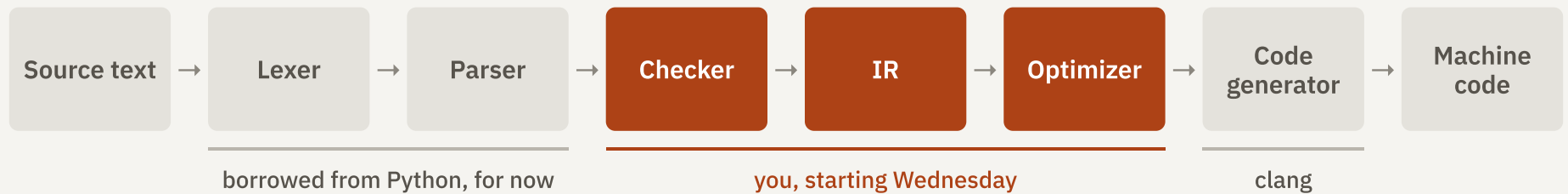
Compiler

reads the program and ***writes another program.***

program → program → answer

Slide 8 was this difference: 19 seconds versus 0.14. We'll use both — you write the compiler; we give you an interpreter that defines the right answers.

The map, with your part highlighted.



By week 10 only the last box is still borrowed.

Wednesday: your first compiler.

Reading: EaC Ch. 1 (today), 4.1–4.3 (Wed).

PA1 goes out Wednesday, due Fri 10/9.

Bring a laptop Thursday to section — not Wednesday to lecture.